🧠 C Programming: Deep Dive into Memory & I/O

To master C for a lifetime, you must understand that the computer only sees **bits**. The "Data Type" is just a lens we use to look at those bits.

🔍 1. The Anatomy of `printf()` (The Interpreter)

`printf` doesn't just "print"; it **formats**. It takes raw data and converts it based on the **Format Specifier**.

The "Mismatch" Trap

If you store a number as an `int` but print it as a `%f`, it doesn't just add `.0`. It looks at the 4 bytes of the integer and tries to decode them using the IEEE 754 Floating Point standard. *Result:* Usually `0.000000` or a very strange number.

Common Specifiers & Their Secret Behaviors:

Specifier	Type	Secret Detail
`%d`	Signed Int	Interprets the first bit as a +/- sign.
`%u`	Unsigned Int	Treats all bits as part of the number (no negatives).
`%c`	Character	Maps the integer value to the ASCII table.
`%p`	Pointer	Prints the actual **Hexadecimal memory address**.
`%0.2f`	Float	Rounds the display to 2 decimal places (doesn't change the actual value).
`%x`	Hexadecimal	Great for seeing how bits are grouped in sets of 4.

📥 2. The Mechanics of `scanf()` (The Memory Writer)

`scanf` requires the **Address-of Operator (`&`)**. *Why?* Because `scanf` is a function. In C, functions usually get a *copy* of data. To change the original variable, `scanf` needs to know exactly where it lives in the RAM.

The "Buffer" Trap

When you type `10` and press **Enter**, you are actually sending three things to the input buffer: `1`, `0`, and `\n` (newline).

1. `scanf("%d", &num);` grabs the `10`.
2. The `\n` stays in the buffer.
3. If the next command is `scanf("%c", &ch);`, it will grab that `\n` immediately, and your program will seem to "skip" your input!

The Fix: Use a space in the specifier: `scanf(" %c", &ch);`. That space tells C to skip any whitespace/newlines left in the buffer.

📜 3. Type Promotion & The "Bucket" Analogy

Think of data types as buckets of different sizes.

Promotion (Small to Big)

If you add a `char` (1 byte) to an `int` (4 bytes), C "promotes" the char. It creates a temporary 4-byte version of the char so the math matches. No data is lost.

Demotion (Big to Small - DANGEROUS)

```
int big = 300;
char small = (char)big;
```

An 8-bit `char` can only hold up to 255 (unsigned) or 127 (signed). `300` in binary is `100101100`. The `char` bucket only has 8 slots, so it **chops off** the leftmost bit. You are left with `00101100` (which is 44). This is **Data Loss**.

⚡ 4. Advanced Operators: The "Short-Circuit" Logic

In exams, you will see expressions like: `if (a > 0 || ++b > 0)`

If `a` is `5` (True), C **never executes** `++b`.

- If you check `b` after this line, it will be its **original value**.
- This is a favorite "trick" question in PGEE. Always check if the first part of an `||` is true or the first part of an `&&` is false.

🛠️ 5. Memory Representation (The "A" Example)

Let's look at the character `'A'` in three ways:

1. **As a Char (`%c`):** `A`
2. **As an Int (`%d`):** `65`
3. **As Binary:** `01000001`

If you do `'A' << 1` (Bitwise left shift), you get `10000010` which is `130`. If you store `130` back into a `signed char`, it wraps around to `-126`.

🚀 Pro-Tip for Lifetime Memory

Whenever you see a C variable, ask yourself:

1. **How many bytes?** (Size)
2. **Signed or Unsigned?** (Does the first bit mean "negative"?)
3. **What is the address?** (Use `&variable` to find out).